

Design and Implementation of an Autonomous Line-Following Robot Using a PD-Controlled Differential Drive Architecture

Applied Automatization — Final Technical Report

Faculty of Engineering, University of Debrecen

Team Leader — Lead Programmer

Name: **Mustafa Ali Mohsin Albusultan**

Mechanical Build Engineer

Name: **Muhammad Salihu**

Electronics & Power Specialist

Name: **Ahsan Muhammad**

Systems Testing & Documentation

Name: **Abdul Fathah**

Supervisor: **Dr. Husam Abdulkareem Almusawi**

Source code repository: <https://github.com/Mustafa-8/Autonomous-Line-Follower-Mechatronics>

Abstract — This paper presents the design, implementation, and validation of an autonomous line-following vehicle developed as a scaled model of an industrial automated guided vehicle. The system integrates a custom 3D-printed chassis, a five-element TCRT5000 reflective infrared sensor array, a TB6612FNG dual-channel motor driver, and two N20 micro metal gear motors under the control of an Arduino Nano executing a Proportional-Derivative (PD) control algorithm. The controller is structured around a four-state supervisory state machine that adapts baseline motor commands to the local track geometry, enabling distinct behavioural regimes for straight, curved, and sharp-turn traversal, together with a deterministic memory-based recovery strategy for line-loss events. A central design contribution is the use of slotted mounting features that permit assembly-time adjustment of sensor working height without chassis modification. Validation testing confirmed repeated successful traversal of complete laps, with three residual failure modes identified and attributed to a single geometric constraint — the fixed battery mounting that limits achievable sensor working distance. The work demonstrates that systematic software compensation of hardware non-idealities can recover much, though not all, performance lost to geometric constraints.

Index Terms — Line-following robot, PD control, differential drive, state machine, embedded systems, automated guided vehicle, mechatronics design.

I. INTRODUCTION

Autonomous guided vehicles (AGVs) are one of the most widely deployed forms of industrial automation, performing material-transport tasks in warehouses, manufacturing plants, and logistics centres. Among the available navigation strategies, line-following remains the most prevalent due to its low infrastructure cost, deterministic behaviour, and ease of reconfiguration. A painted or taped path on the floor is sufficient to define an entire transport route, eliminating the need for costly positioning infrastructure such as LiDAR mapping or vision-based localisation.

This project investigates the design, construction, and validation of a small-scale autonomous line-following robot as a representative model of an industrial AGV. The objective is to demonstrate, at an accessible scale, the same closed-loop sensing-and-actuation principles that govern full-size automation systems: a sensor subsystem perceives the environment, a control algorithm computes a corrective response, and an actuator subsystem executes that response.

A. Problem Statement and Constraints

The central problem is to develop a mobile platform that autonomously tracks a black line on a white surface while satisfying three competing requirements: accuracy (minimal lateral deviation), stability (no sustained oscillation), and robustness (recovery from temporary line loss without operator intervention).

These objectives are pursued under three physical constraints. First, the TCRT5000 reflective infrared sensors operate reliably only within a 2–4 mm working window above the floor. Second, a 15 mm thick lithium-ion battery is mounted beneath the chassis to centralise mass, imposing a minimum ground clearance and consequently a fixed chassis height. Third, the TB6612FNG motor driver occupies the Arduino's hardware interrupt pins (D2, D3), constraining future encoder integration to alternative interrupt mechanisms.

B. Objectives

- 1) Design a chassis that resolves the geometric conflict between battery clearance and sensor working distance without requiring iterative re-fabrication.
- 2) Implement a closed-loop PD controller capable of tracking straight segments, gradual curves, and sharp turns.
- 3) Develop a state-based supervisory layer that adapts baseline speed to local track geometry.
- 4) Implement a deterministic line-loss recovery strategy.
- 5) Compensate for hardware asymmetries entirely in software.

The remainder of this paper is organised as follows. Section II develops the mathematical model. Section III details the mechanical design. Section IV describes the hardware. Section V presents the control system architecture. Section VI documents the software. Section VII reports validation testing. Section VIII evaluates the results, and Section IX concludes.

II. MATHEMATICAL MODELING

This section establishes the mathematical foundation governing the robot's motion and the closed-loop control law used to regulate it.

A. Differential-Drive Kinematics

Let v_L and v_R denote the linear velocities of the left and right wheels, and let L denote the wheelbase. The vehicle's linear velocity v and angular velocity ω are given by:

$$v = (v_R + v_L) / 2 \quad (1)$$

$$\omega = (v_R - v_L) / L \quad (2)$$

Equation (2) is the key relationship exploited by the controller: a single correction signal applied with opposite sign to the two wheels produces pure rotation without changing forward speed.

B. Sensor Model and Weighted Error

The sensor array comprises five TCRT5000 reflective infrared modules in a transverse linear arrangement, each producing a binary output. Let $s_i \in \{0, 1\}$ denote the reading of sensor i , with weights $w_i \in \{-4, -2, 0, +2, +4\}$ representing lateral offset from the vehicle centreline. The instantaneous line-position error is:

$$e(t) = \sum w_i \cdot s_i / \sum s_i, \quad \sum s_i > 0 \quad (3)$$

The error is signed: negative values place the line to the left, positive to the right. The denominator interpolates when multiple sensors are active, bounding $e(t) \in [-4, +4]$. The case $\sum s_i = 0$ is handled by the recovery rule of (5).

C. Proportional-Derivative Control Law

The continuous-time PD law is:

$$u(t) = K_p \cdot e(t) + K_d \cdot de(t)/dt \quad (4)$$

Because the controller runs on a discrete-time microcontroller with effectively constant loop period, the derivative is approximated by a backward difference and Δt absorbed into K_d :

$$u_k = K_p \cdot e_k + K_d \cdot (e_k - e_{k-1}) \quad (5)$$

The integral term is deliberately omitted. Persistent steady-state error in lateral position is not a meaningful failure mode in line-following — the line itself defines the reference — and integral action introduces windup risk during line-loss events. The correction u_k is applied symmetrically to baseline wheel commands b_L and b_R :

$$v_L^{cmd} = b_L + u_k, \quad v_R^{cmd} = b_R - u_k \quad (6)$$

Both commands are saturated to the actuator's PWM range $[-255, +255]$.

D. Line-Loss Recovery

When no sensor detects the line, the controller substitutes a deterministic recovery error derived from the sign σ of the most recent non-zero observation:

$$e_k^{eff} = \sigma \cdot e_{max}, \quad e_{max} = 4 \quad (7)$$

Critically, e_{k-1} is *not* updated during recovery iterations; this prevents a derivative spike when the line is reacquired.

E. Gain Tuning

The gains were determined experimentally through the manual variant of the Ziegler-Nichols procedure: K_p was raised from zero until sustained oscillation appeared, then reduced slightly, after which K_d was raised to damp the residual oscillation. The final values $K_p = 12$ and $K_d = 14$ were selected as the best compromise between accuracy on straights, stability on curves, and aggressiveness in recovery. The slightly higher derivative gain reflects a deliberate preference for damping, appropriate given the vehicle's inertia and the limited tyre grip on the test surface.

III. MECHANICAL DESIGN

The mechanical design was driven by three requirements: accommodation of the drive system within a compact footprint, adequate ground clearance for the under-mounted battery, and reconciliation of the resulting chassis height with the strict working distance of the optical sensors.

A. Chassis Geometry and Fabrication

The chassis is a single-piece component designed in Autodesk Fusion 360 (with initial 2D drafting in AutoCAD) and fabricated by FDM 3D printing, with a uniform thickness of 2.7 mm. The overall envelope is 155 mm × 115 mm. Rather than a simple rectangle, the chassis is a stepped polygon comprising three regions: a narrow rear section (75 mm wide) housing the power switch and battery-holder mounts, a wide central section (115 mm) accommodating the motor brackets and wheels, and a narrow forward section (80 mm) carrying the sensor board.

The stepped geometry minimises material consumption while preserving structural rigidity at the load-bearing centre, and reduces the moment of inertia about the vertical axis, improving rotational responsiveness.

B. Drive System and Wheel Configuration

Two N20 micro metal gear motors (6 V, 500 RPM, integrated quadrature encoders) are mounted to the underside of the chassis via brass L-brackets and stainless M3 × 16 mm screws. Wheels are 42 mm diameter rubber-tyred, providing an axle height of approximately 21 mm and a battery-to-floor clearance of approximately 6 mm. A steel ball castor at the leading edge completes the three-point ground support.

C. The Geometric Conflict and Its Partial Resolution

The most demanding constraint arose from the conflict between the chassis height (fixed at 21 mm) and the TCRT5000 working distance (2–4 mm). Two solutions were evaluated: redesigning the chassis with a lowered forward extension (rejected due to fabrication cost), and suspending the sensor board through *slotted* mounting features (selected). Four sensor-board mounting holes are obround slots measuring 3.2 mm × 5 mm with 1.6 mm corner radii, permitting fastener position — and hence vertical standoff length — to be set at assembly time.

In practice, the achievable sensor height is bounded above by a third constraint not initially foreseen: the battery is fixed beneath the chassis by two M3 screws engaging from above and cannot be relocated without invalidating the chassis design. Lowering the sensor board further would require the standoffs to extend below the bottom plane of the battery, which the geometry does not permit. The as-built sensor height after final assembly and castor-tilt adjustment is approximately 15 mm — substantially above the TCRT5000's nominal working window. The resulting design does not fully resolve the original conflict; it represents a constrained compromise within the fixed mechanical envelope. The consequences are characterised in Sections VII and VIII.

D. Mounting Slots and Fastener Strategy

All M3 fastener mounting features are obround slots of identical geometry (3.2 mm × 5 mm, 1.6 mm radius), providing both a clearance fit for M3 hardware and up to 2 mm of positional adjustment. The chassis contains 16 fastener slots and one 21 × 10 mm cut-out, summarised in Table I.

TABLE I · Chassis Fastener Inventory

Group	Qty	Fastener	Purpose
Left motor	2	M3×16 steel	Alignment
Right motor	2	M3×16 steel	Alignment
Main board	4	M3×16 steel	Electronics
Sensor board	4	M2×16 nylon	Height adjust
Battery	2	M3×6 steel	Cell retention
Castor	2	M3×22, 6 nuts	Height jacking

Fastener material is selected by location. Stainless steel is used wherever the joint experiences mechanical stress (motor brackets, electronics board, battery holder); nylon M2 hardware is retained on the sensor board because metal screws in close proximity to the IR LED and phototransistor pads would create short-circuit risk. The castor mount uses six stacked nuts on each M3 × 22 mm screw, forming a threaded jacking system that permits independent height adjustment of the front of the chassis. This was actively used during assembly to bring the sensors as close to their working window as the battery clearance permits.

Fig. 1. Chassis 2D outline with dimensions (Fusion 360 drawing).

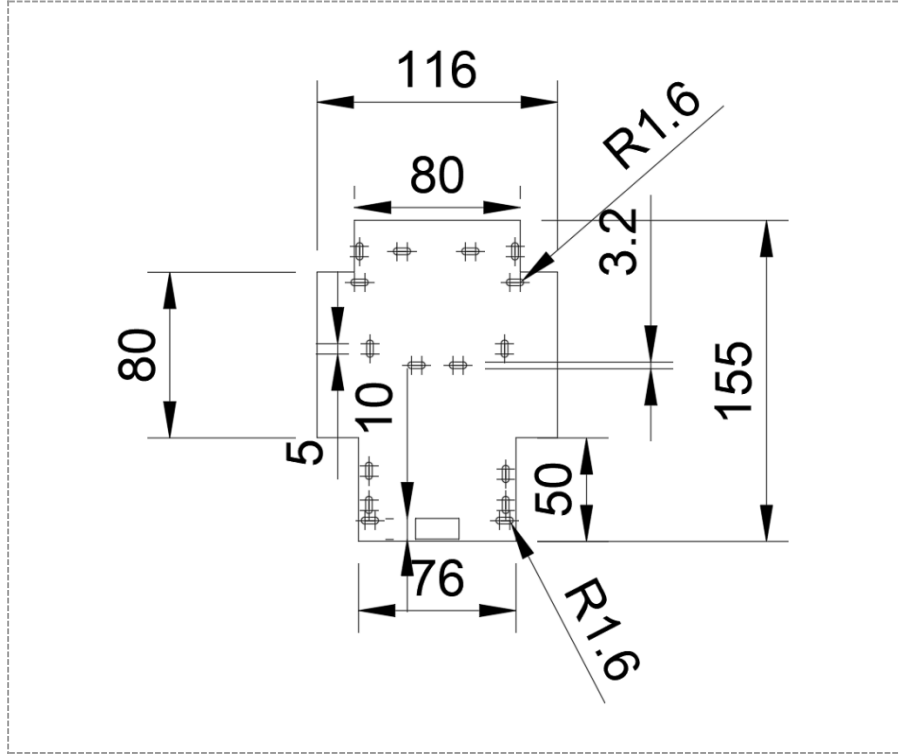
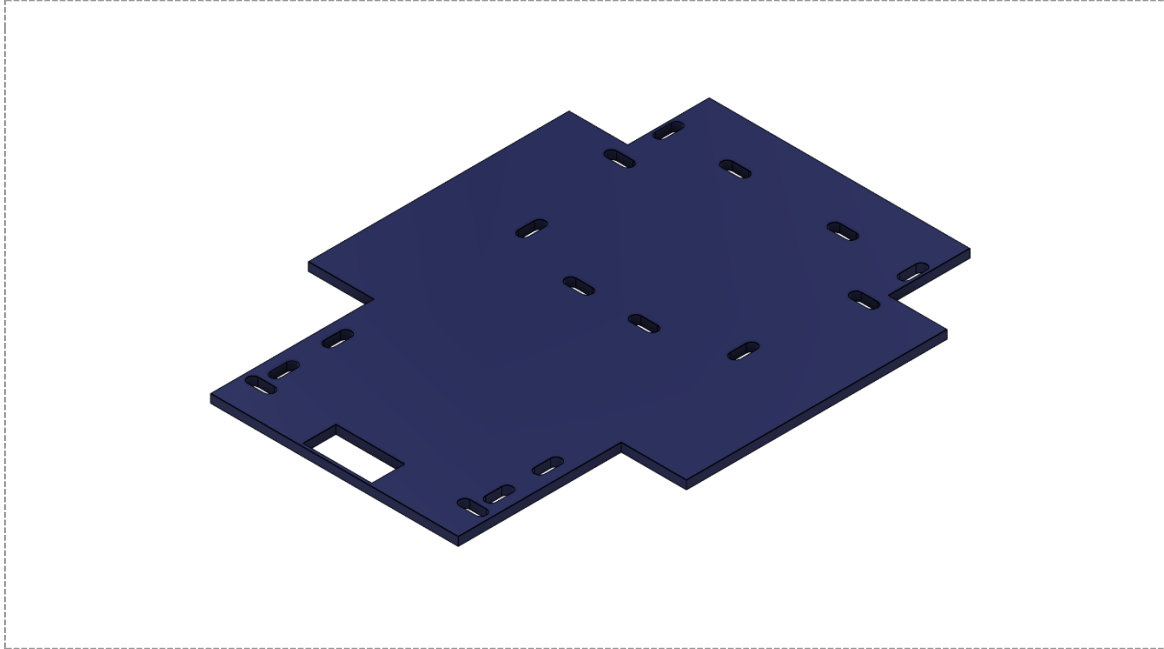


Fig. 2. Fusion 360 isometric view of the assembled chassis model.



IV. ELECTRICAL AND HARDWARE SYSTEM

The electrical subsystem comprises four functional layers: sensing, control, actuation, and power conditioning. Each is implemented with off-the-shelf modular components. The system-level wiring is shown in Fig. 4; the bill of materials is summarised in Table II.

A. Microcontroller and Sensing

The control layer is implemented on an Arduino Nano (ATmega328P) selected for its compact footprint, sufficient pin count, and direct Arduino IDE compatibility. The Nano is powered from the regulated 5 V rail rather than via its onboard linear regulator, avoiding thermal dissipation across the 7.4-to-5 V drop.

The sensing layer consists of five TCRT5000-M reflective infrared modules in a transverse linear array, each producing a *digital* output (logic 1 = black line, logic 0 = white background) via an onboard LM393 comparator with a sensitivity-trimming potentiometer. Digital outputs were chosen over analog mode for noise immunity and to avoid ADC overhead in the control loop; the loss of intra-sensor resolution is compensated by the weighted-average error formulation of Section II-B. The five outputs are routed to Arduino pins A0–A4 (used as digital GPIO).

B. Actuation

The TB6612FNG dual H-bridge driver controls the two N20 motors, each rated 6 V and 500 RPM with integrated quadrature encoders. Driver pin allocation is shown in Table III. Pins D3 and D5 carry the PWM signals because they tie to ATmega328P timers 2 and 0, providing hardware-generated 490 Hz PWM. A consequence is that the Arduino's hardware interrupt pins (D2, D3) are occupied, forcing the encoder lines to D9–D12 and any future encoder counting onto Pin Change Interrupts.

TABLE II · System Hardware Summary

Layer	Component	Role
Sensing	5× TCRT5000-M	Binary line detection
Control	Arduino Nano	PD loop, state machine
Actuation	TB6612FNG + 2× N20	Differential drive
Power	2× 18650 + LM2596S	7.4 V motor, 5 V logic

TABLE III · TB6612FNG Pin Assignment

Arduino	Driver	Function
D6	STBY	Driver standby (HIGH = enabled)
D3	PWMA	Right motor speed (PWM)
D7	AIN1	Right motor direction
D8	AIN2	Right motor direction
D5	PWMB	Left motor speed (PWM)
D4	BIN1	Left motor direction
D2	BIN2	Left motor direction

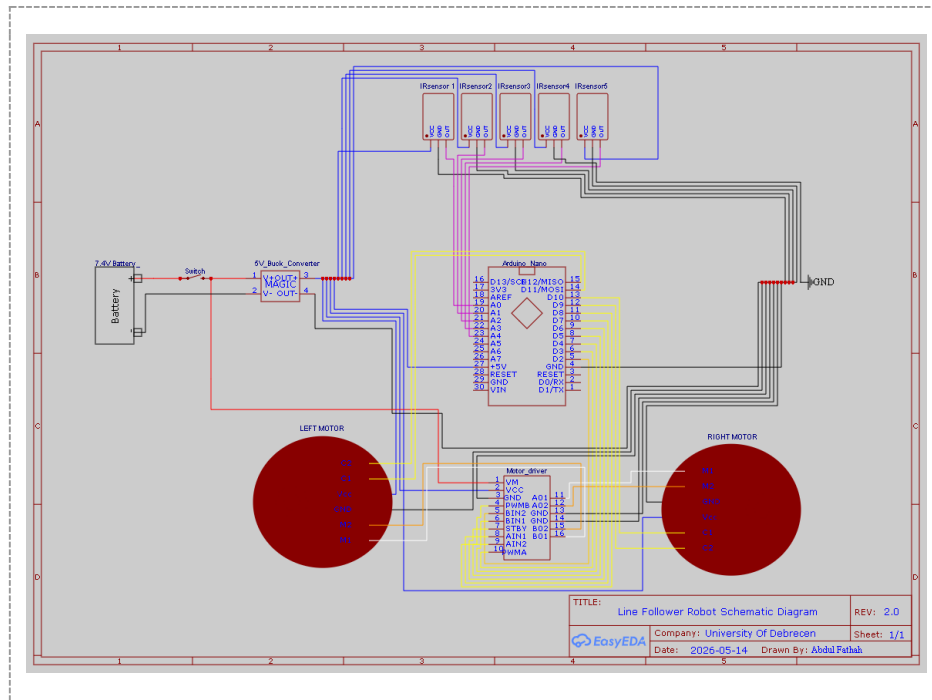
C. Power System

Two protected 18650 lithium-ion cells in series provide a nominal 7.4 V rail (fully charged: 8.4 V; cutoff: ≈ 6 V). A LM2596S adjustable buck converter steps this down to a calibrated 5.0 V logic rail, verified to within ± 0.05 V across the full discharge range. The switching topology was selected over linear regulation for its $>80\%$ efficiency, which directly extends battery run-time given the continuous draw of the five sensor IR LEDs. A single common-ground node is shared by the battery negative, regulator ground, microcontroller, motor driver (logic and power grounds), and sensor modules — eliminating ground loops and ensuring consistent reference for the digital sensor signals.

D. Reserved Encoder Inputs

The N20 encoder channels (C1 and C2 of each motor) are wired to Arduino pins D9–D12 but are not used by the current firmware. These pins were chosen because the hardware interrupt pins (D2, D3) were unavailable. A future firmware revision could implement Pin Change Interrupt handlers on Port B to enable an inner velocity control loop and odometric position estimation.

Fig. 3. Electrical wiring schematic.



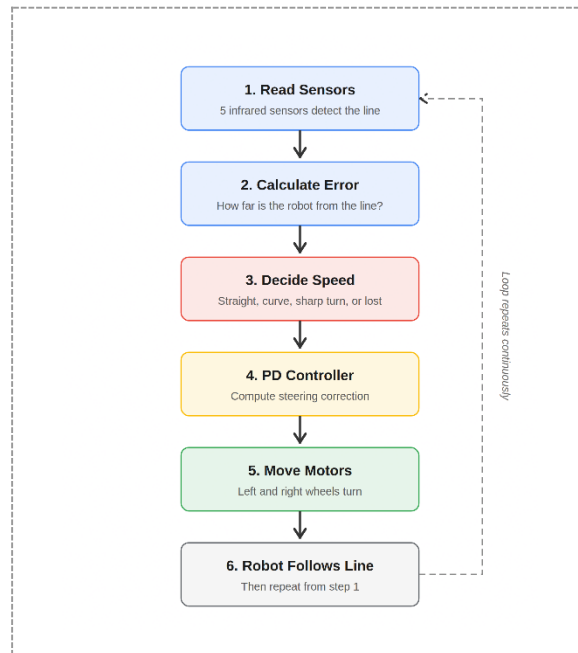
V. CONTROL SYSTEM AND ALGORITHM DESIGN

The control architecture is hierarchical: an inner PD feedback loop performs continuous error regulation, an outer supervisory layer adapts the baseline operating speed to local track geometry, and a recovery layer handles line-loss events. The complete signal flow is shown in Fig. 5.

A. Architecture Overview

Five binary sensor signals are read each iteration and combined into a weighted-average error via (3). The error feeds two parallel blocks: a state classifier that selects the operating mode, and the PD controller that computes the correction signal. The state index drives a baseline lookup; the PD output is then added differentially to those baselines. The result is saturated to $[-255, +255]$ and forwarded to the motor driver. This separation of concerns — fine steering via PD, coarse speed regime via state machine — permits independent tuning of the two axes.

Fig. 4. Control system block diagram.



B. Supervisory State Machine

The four-state supervisory layer modulates the PD controller's authority by changing the baseline against which corrections are applied. The states and their commands are shown in Table IV.

TABLE IV · State Machine Baseline Commands

State	Condition	b_R	b_L
STRAIGHT	$ e = 0$	220	211
CURVE	$1 \leq e \leq 2$	160	151
SHARP	$ e > 2$	90	81
LOST	$\Sigma s_i = 0$	70	61

The behavioural significance lies in the relative magnitude of the baseline and the PD correction. In STRAIGHT, the baseline of ~ 215 dominates and a maximum correction produces gentle steering at high speed. In SHARP, the baseline of ~ 85 is comparable to the correction, so a maximum error commands one wheel toward zero or reverse — producing a *pivoting* turn rather than an arc. The state machine thus does not merely slow the vehicle; it changes the kinematic character of motion to match the track geometry.

The 9-unit offset ($b_L = b_R - 9$) applies uniformly across all four states and compensates for the empirically measured speed asymmetry between the left and right N20 motors. Embedding the offset at the baseline level keeps the PD gains physically meaningful as pure steering quantities.

C. Line-Loss Recovery

The LOST state is functionally distinct from the others: the error must be *synthesised* because no measurement is available. The variable lastTurnDir tracks the sign of the most recent non-zero error. When all sensors read zero, the controller treats the error as saturated to ± 4 in the direction of lastTurnDir, forcing maximum corrective action. Three properties of this design are noted: (i) e_{k-1} is deliberately not updated during recovery, preventing a derivative spike when the line is reacquired; (ii) the LOST baseline (70/61) is intentionally low so the recovery is a rotational search rather than a forward sweep; and (iii) the strategy is *memory*-based rather than *search*-based — the system commits to the last-known direction, on the well-founded assumption that the line was lost because the vehicle was turning toward it.

VI. SOFTWARE ARCHITECTURE

The firmware is written in C++ targeting the Arduino framework, compiled with avr-gcc and uploaded to the Nano via its USB-to-serial bridge. It consists of a single translation unit containing setup(), loop(), and a setMotors() helper.

A. Program Structure and Memory Discipline

Pin numbers, sensor readings, and the weights array are declared as uint8_t or int8_t to reduce SRAM footprint; the ATmega328P provides only 2 KB. The setup() function configures pins, drives all motor pins to a safe stopped state *before* raising STBY (per the TB6612FNG datasheet recommendation), then waits 2 s to allow operator placement of the vehicle. The loop() function implements the seven-step control cycle: sensor sampling, error computation, state classification, baseline lookup, PD computation, saturation, actuator update. No delay() calls or busy-waits are used; the iteration rate is dominated by the five digitalRead() calls and reaches several kHz on the ATmega328P — orders of magnitude above the drive system's mechanical bandwidth.

B. Software-Level Hardware Calibration

Three calibration decisions are implemented entirely in software. First, the 9-unit PWM offset ($b_L = b_R - 9$) compensates for the left motor's higher angular velocity at equal PWM duty. Second, the BIN1/BIN2 direction pins are inverted relative to AIN1/AIN2 in setMotors() to compensate for the left motor's opposite mounting orientation. Third, the per-sensor comparator thresholds are tuned via the modules' onboard potentiometers against the actual installed sensor height and test surface. These three calibrations together embody a consistent design philosophy: hardware non-idealities are compensated in software, since software adjustments can be re-tuned without disassembly and the firmware serves as a single authoritative record of all calibration values.

C. Key Code Excerpts

The core sensor-reading and error-computation loop:

```
for (uint8_t i = 0; i < 5; i++) {
    s[i] = digitalRead(SENSOR_PINS[i]);
    if (s[i]) { sum += WEIGHTS[i]; count += 1; }
}
if (count > 0) {
    error = sum / count;
    // state classification ...
```

```

if (error < 0) lastTurnDir = -1;
else if (error > 0) lastTurnDir = 1;
} else {
    error = (lastTurnDir < 0) ? -4
           : (lastTurnDir > 0) ? 4 : lastError;
    turnType = 3; // LOST
}

```

The PD computation, with conditional update of lastError:

```

int derivative = error - lastError;
int correction = (Kp * error) + (Kd * derivative);
if (lineDetected) lastError = error;
int leftSpeed = constrain(baseLeft + correction,
                          -MAX_SPEED, MAX_SPEED);
int rightSpeed = constrain(baseRight - correction,
                           -MAX_SPEED, MAX_SPEED);
setMotors(leftSpeed, rightSpeed);

```

The setMotors() helper, with channel B direction inverted relative to channel A:

```

void setMotors(int leftSpeed, int rightSpeed) {
    digitalWrite(AIN1, rightSpeed >= 0 ? HIGH : LOW);
    digitalWrite(AIN2, rightSpeed >= 0 ? LOW : HIGH);
    analogWrite(PWMA, abs(rightSpeed));
    digitalWrite(BIN1, leftSpeed >= 0 ? LOW : HIGH);
    digitalWrite(BIN2, leftSpeed >= 0 ? HIGH : LOW);
    analogWrite(PWMB, abs(leftSpeed));
}

```

The complete source listing is available in the project GitHub repository [9].

VII. TESTING AND VALIDATION

Testing was conducted in two phases: bench testing of individual subsystems, and on-track validation of full-lap runs. The methodology was primarily qualitative — focused on identifying and characterising behavioural failure modes — consistent with the project's objective of demonstrating control principles.

A. Bench Testing

Each TCRT5000 module was verified against the actual track surfaces at the installed working distance, with its potentiometer trimmed to centre the comparator threshold between the two reflectance levels. Motor polarity and asymmetry were verified with the vehicle suspended, confirming the channel-B direction inversion and the +9 PWM offset compensation. The LM2596S output was measured at $5.0\text{ V} \pm 0.05\text{ V}$ across the battery's full discharge range. Common-ground continuity was verified across all subsystems.

B. Track Validation and Successful Behaviour

On-track testing used a closed track consisting of an approximately 18 mm wide black line on a light-coloured background, with straight segments, gradual curves, and turns approaching 90° . The vehicle successfully completed multiple full-lap runs. On straights and gradual curves, the PD controller produced smooth corrections with no sustained oscillation; the 9-unit asymmetry compensation eliminated heading drift at zero correction. State transitions between STRAIGHT and CURVE were visibly smooth. The line-loss recovery activated multiple times — most often during sharp turns — and reacquired the line on every observed occurrence, validating the deterministic memory-based recovery strategy.

C. Identified Failure Modes

Three behavioural limitations were observed.

- 1) *Sharp 90° turns* were handled inconsistently. On some passes the vehicle pivoted correctly; on others it overshoot and recovered via the LOST-state rotation. The root cause is the elevated sensor working distance ($\sim 15\text{ mm}$ instead of $2\text{--}4\text{ mm}$), which enlarges the IR illumination spot and blurs the detection edge, delaying corner detection until the vehicle has already passed the apex.
- 2) *Ambient lighting sensitivity*. The TCRT5000 is an active sensor and is in principle insensitive to ambient visible light, but at the elevated working distance the reflected signal is attenuated and sits closer to the comparator noise floor. Small ambient IR contributions then become significant relative to the reflected signal, producing occasional false detections under reduced room lighting.
- 3) *Small oscillation on gradual curves*. The integer error signal (-4 to $+4$) has a discretisation step that, at the installed height, produces a small dead zone around the line centreline as the IR coverage regions of adjacent sensors widen and lose crisp boundaries.

All three failure modes share a common root cause: the elevated sensor working distance imposed by the unresolved geometric constraint of Section III-C. The control algorithm is not deficient; the *signal* delivered to it is degraded.

D. Software-Level Mitigations

The mitigation strategy was software-only: the per-sensor comparator thresholds were re-tuned against the actual installed height; the PD gains were refined to provide additional damping; and the state-machine baseline speeds were reduced to limit corner-engagement inertia. These adjustments do not eliminate the underlying limitation but reduce its operational impact to the point at which repeatable lap traversal becomes achievable under typical room lighting.

VIII. RESULTS AND EVALUATION

The vehicle is evaluated against the five objectives of Section I-B and against its closed-loop performance during validation.

A. Achievement of Objectives

Of the five stated objectives, four were achieved in full and one — resolution of the battery-clearance / sensor-height conflict — was partially achieved. The slot mechanism that constitutes the design contribution functions as intended; the residual limitation is that the achievable sensor height is bounded above by the battery's fixed mounting, a constraint not foreseen at the design stage. The PD controller, state machine, line-loss recovery, and software-only hardware compensation all perform as specified.

B. Closed-Loop Performance

Straight-line tracking is the most robust regime: zero PD correction is needed at steady state and the asymmetry compensation eliminates drift. Gradual curves are negotiated smoothly with no observable oscillation. Sharp turns engage the SHARP state and the vehicle's motion visibly transitions from arcing to pivoting; when corner detection occurs in time, the manoeuvre is executed correctly. Line-loss recovery functioned reliably on every observed activation with no overshoot at reacquisition — validating the deliberate decision not to update lastError during recovery.

C. Constraint-Bound Limitations

The residual limitations are *not* evidence of controller deficiency; they reflect the elevated sensor working distance imposed by the geometric constraint. Further tuning of K_p , K_d , or the state baselines would yield diminishing returns. Addressing the sensor height directly — through a redesigned chassis or a different battery form factor — would address the root cause and eliminate all three failure modes simultaneously.

D. Overall Evaluation

The vehicle demonstrates the required closed-loop control principles successfully. It completes lap traversal repeatedly, executes all four state regimes visibly and correctly, and recovers from line-loss events deterministically. Its limitations concentrate into a single, well-characterised root cause, providing a clear actionable target for the next design iteration rather than a diffuse set of unrelated issues. This concentration of residual issues is itself a positive evaluation outcome.

IX. CONCLUSION

A. Summary

This work presented the design, fabrication, and validation of an autonomous line-following vehicle. The system integrates a custom 3D-printed chassis with adjustable slotted mounting features, a five-element TCRT5000 sensor array, a TB6612FNG dual H-bridge driver with N20 gear motors, and an Arduino Nano executing a tuned PD controller wrapped in a four-state supervisory layer. The vehicle achieves repeated successful traversal of complete laps under typical operating conditions and validates the design philosophy of compensating hardware non-idealities entirely in software.

B. Challenges and Limitations

The principal challenge was a geometric constraint that emerged at the integration stage: the fixed battery mounting beneath the chassis prevented the slot mechanism from bringing the sensors fully into their nominal 2–4 mm working window. The as-built sensor height of approximately 15 mm gives rise to three residual failure modes, all attributable to reduced reflected-signal-to-noise ratio. Software mitigations reduced the operational impact but did not eliminate the underlying signal-quality limitation. This experience reinforces the principle that not every hardware constraint can be fully resolved in software, and that the value of an honest evaluation lies in identifying the boundary between what software compensation can and cannot achieve.

C. Future Work

Three directions for future development are identified, in order of expected impact.

- 1) *Resolution of the sensor-height constraint.* Two paths are available: adopting an alternative battery form factor (e.g., a thinner lithium-polymer pouch cell of equivalent capacity), or redesigning the chassis so that the battery is mounted in a region that does not conflict with sensor placement. Either path would address the three failure modes of Section VII simultaneously.

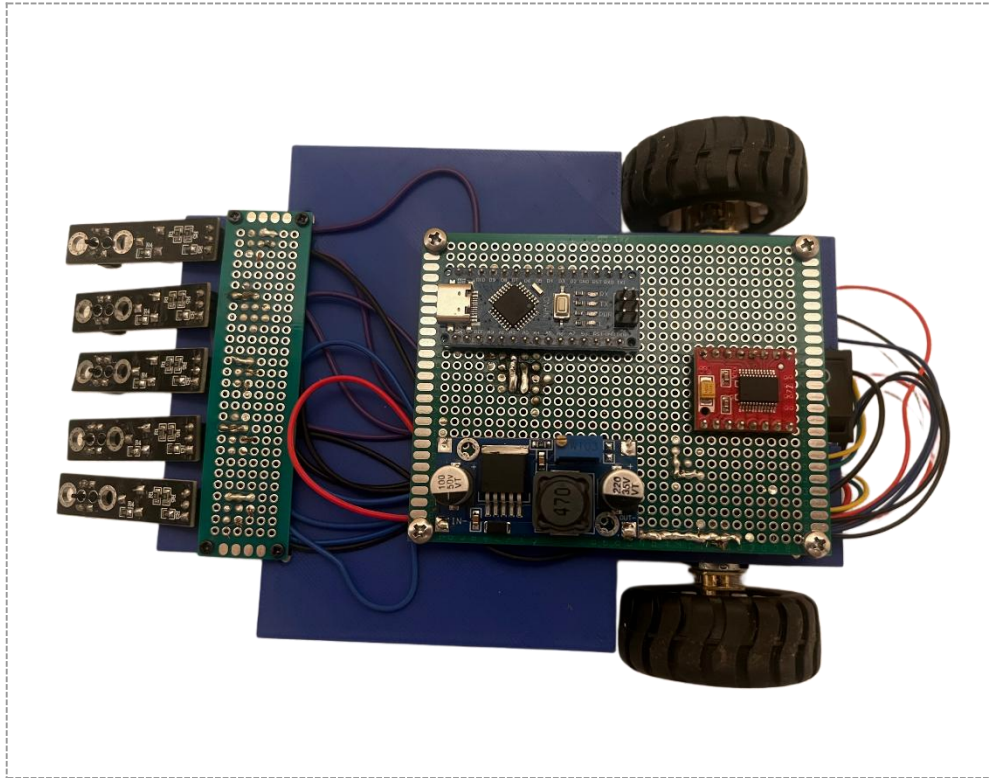
- 2) *Closed-loop velocity control via encoder feedback.* The N20 encoders are physically wired to pins D9–D12 but unused by the current firmware. A future firmware revision can implement Pin Change Interrupt handlers on Port B to enable an inner velocity loop and odometric position estimation.
- 3) *Extended state machine for intersection handling.* Specific multi-sensor patterns retained in the per-iteration sensor array could be used to recognise intersections, end-of-track markers, and perpendicular crossings, moving the vehicle toward the limited topological navigation behaviour of commercial AGVs.

D. Closing Statement

The vehicle described in this report meets its functional objective and demonstrates the principles of closed-loop sensor-driven control that constituted the project's educational purpose. Its residual limitations are documented transparently and attributed to a single, well-characterised cause, providing a clear basis for the design improvements outlined above.

Appendix. PHOTOGRAPHIC DOCUMENTATION

Fig. 5. Photograph of the assembled vehicle.



References.

- [1] Toshiba Semiconductor, "TB6612FNG: Dual H-Bridge Motor Driver IC Datasheet," Toshiba Corporation, Tokyo, Japan.
- [2] Vishay Semiconductors, "TCRT5000 / TCRT5000L: Reflective Optical Sensor with Transistor Output," Vishay Intertechnology, Inc.
- [3] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA: MIT Press, 2011.
- [4] J. G. Ziegler and N. B. Nichols, "Optimum settings for automatic controllers," *Transactions of the ASME*, vol. 64, pp. 759–768, 1942.
- [5] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ: Prentice Hall, 2010.
- [6] Atmel Corporation, "ATmega328P 8-bit AVR Microcontroller Datasheet," 2015. (Now Microchip Technology Inc.)
- [7] Arduino, "Arduino Nano Product Reference," Arduino S.r.l., <https://docs.arduino.cc/hardware/nano>.
- [8] Texas Instruments, "LM2596 SIMPLE SWITCHER Power Converter 150-kHz 3-A Step-Down Voltage Regulator Datasheet," Texas Instruments Inc.
- [9] M. A. M. Albusultan, "Autonomous Line Follower Mechatronics — Source Code Repository," GitHub, 2026. [Online]. Available: <https://github.com/Mustafa-8/Autonomous-Line-Follower-Mechatronics>